

長期アプリ開発における チーム連携の成果

Team3 開発報告

INIAD Student

2026-01-11

—

長期アプリ開発におけるチーム連携の成果

👥 「アプリの機能」ではなく「連携の工夫」に注目 🙌

🕒 4 人チーム / 約 8 か月（長期プロジェクト）

⚠️ 技術的課題以上に「人の認識ズレ」や「作業の偏り」が課題

🏰 メンバーのやる気に頼らず、**「破綻しにくいチーム構造」**を作る

導入

—

今回の発表では、
アプリの機能や技術的な凄さではなく、
「4 人で長期間開発を続けるために、どんな連携の工夫をしたか」
に焦点を当てて話します。
この演習は期間が長く、
技術的な問題以上に、
人の認識ズレや作業の偏りが起きやすいと感じました。
そこで私たちは、
メンバーのやる気や能力に頼るのではなく、
破綻しにくいチーム構造を作ることを意識しました。
このあと、スケジュールに沿って、
人間的な連携と、開発上の連携の両面から説明します。

技術選定と合意形成

技術選定と合意形成

—

初期段階で“唯一の真実”（SSOT）を決める

- docs/ ディレクトリを作成
- 合意形成と **同時に** 作成
- 「後から書く」を禁止

効果: 議論が蒸し返されない。
「docs に書いてある」で終了。

```
docs/  
├── 00_OVERVIEW.md  
├── 01_ARCHITECTURE.md  
├── 02_DOMAIN_KNOWLEDGE.md  
├── 03_DECISION_LOG.md  
└── CONTRIBUTING.md
```

3 / 8

技術選定と合意形成

— ドキュメントによる合意形成

企画が固まった段階で、開発に入る前に **docs ディレクトリ** を作りました。中には、

- 技術構成
 - アプリの目的と価値
 - コーディング規約
- をまとめました。

特に意識したのは、**ドキュメントを「後から書く説明書」にしないこと**です。

勉強会の後に時間を取り、事前に用意した質問テンプレを使って3人で議論しながら回答してもらい、私がその場で文章にまとめました。

こうすることで、「誰かが勝手に決めた仕様」ではなく、**全員が納得した判断を、そのまま文章にしたもの**になりました。このドキュメントを「**唯一の真実**」として扱ったことで、初期の無駄な議論や認識ズレを減らすことができました。

チーム運営の工夫

チーム運営の工夫

—

チーム構成の工夫 (2 人 x 2 チーム)

✓ メリット

- 画面共有実況による疲弊を防ぐ
- 普段はペアで完結し高速化

✓ 工夫

- 全員集合は「確認」のみ
- 議論は持ち込まない

チーム運営の工夫

— チーム構成の工夫 (2 人 x 2 チーム)

チームは 4 人でしたが、最初から 4 人全員で常に作業する形は取りませんでした。
理由は、過去のハッカソン経験で、
「作業者は 1 人なのに、3 人が画面共有を見続けて疲弊する」 という状況を何度も経験していたからです。
学生は物理的に集まりにくく、通話を繋がないと作業しない、という前提があります。
その結果、**「とりあえず全員で繋ぐ → 実際に手を動かすのは 1 人」** という非効率が起きやすいと考えました。
そこで、**技術班 2 人・デザイン班 2 人** の 2 チームに分け、普段の作業はこの単位で完結させました。
全員が揃うのは、目的を限定した定例ミーティングのみです。この構成にしたことで、

- 作業中の無駄な同席がなくなり
- 各チームが自律的に進められる

という効果がありました。

短く・目的を限定した全体同期

🕒 毎週日曜 22:00 (Max 30 分)



- 1. 📋 事前: カンバン更新
- 2. 👁 Meet 中:
 - ・ 進捗確認 / 軌道修正 / 質問受付
- 3. 💬 Meet 後:
 - ・ 個別 Slack で深い議論

⌚ 実質 15 分で解散

チーム運営の工夫 — 定例ミーティングの工夫

夏休みのリモート開発では、デザイン班の進捗が見えづらいという問題が出ました。
ただ、ここで長時間の会議を増やすと、先ほど話した「疲弊する構造」に戻ってしまいます。
そこで、**毎週日曜 22 時、30 分未満で必ず終わる定例 Meet**を設定しました。
30 分以内に終わらせるために工夫。
Meet の前に、全員がカンバンを更新します。Meet 中にやることは 3 つだけです。
1. つ目、進捗の確認。
2. つ目、タスクのズレやおかしな点の軌道修正。
3. つ目、質問の一次受付。
議論や作業は一切しません。
質問が深くなりそうな場合は、「**Meet が終わってから個別に Slack で**」と切り分けました。
結果として、実際の Meet は 15 分程度で終わることがほとんどでした。
Meet 後は、それぞれの 2 人チームで集まり直して作業を進めていました。
これにより、全体での情報共有と、各チームの集中作業を両立できました。

まとめ



人に依存しない構造を作る



技術選定

ゼロから揃えて
属人化を排除



ドキュメント

合意形成のプロセス
として利用



構造化

同期を最小化し
自律分散

↓ チームを設計することで連携/開発をスムーズにできる

まとめ

— チーム連携の成果と学び

今回のプロジェクトを通して、チーム連携で重要なのは、仲良く話すことや、全員が常に頑張ることではないと感じました。

人のやる気やスキル差があっても、**破綻しにくい構造を作ること**が大切でした。

チーム構成、定例ミーティングの設計、技術選定や仕様共有の方法など、連携を「設計する」ことを意識しました。

この経験は、今後チームで開発を行う際にも活かせると考えています。